# Aula 10 — Ficha de entrega do exercício final

Oct 13, 2026

## Identificação

Preencha antes de começar a Etapa 0. O professor corrige pelo código do repositório; esta ficha mostra o que você viu rodar.

| Campo | Preencha |
|---|---|
| Nome completo | FELIPE OLIVEIRA GOUVEIA |
| E-mail | felipe.gouveia@pcdf.df.gov.br |
| Repositório Git (link) | --- |

O repositório precisa estar acessível ao professor (público, ou com acesso dado a ele). Não envie o arquivo `.env`

## Como preenchere

Ao terminar cada etapa, cole aqui a saída do terminal e responda às perguntas daquela etapa. Não deixe para o fim.

- **Saídas:** cole o texto do terminal, não um print da tela. Se for longa, cole só as linhas que a pergunta pede. (*o texto ficou desconfigurado, achei melhor o print para melhor vizualização*)
- **Respostas:** até 3 linhas cada, com as suas palavras.
- **Etapa que não terminou:** escreva o que falta e onde travou. Uma etapa incompleta, mas explicada, vale mais do que um campo em branco.
- **Conferir:** no fim de cada etapa, rode `python conferir.py N` e marque o resultado na tabela da última seção.

## Etapa 0: quem verifica cada regra

Leia `dados/normas.md` e diga quem confere cada regra: função Python, agente ou humano. A última coluna você preenche na Etapa 11.

| Regra | Tema | Quem verifica | Por quê | Onde foi implementada |
|---|---|---|---|---|
| N1 | férias | função Python | 3 períodos, cada um com pelo menos 5 dias, e o total não passa | em violacoes_objetivas(). Os dias corridos vêm de resumo_period |

| Regra | Tema | Quem verifica | Por quê | Onde foi implementada |
|---|---|---|---|---|
| | | | de saldo_ferias_dias. | o(). |
| N2 | férias (venda) | função Python (limites) e agente Financeiro (cálculo do valor) | vendidos ≤ 10, gozados + vendidos ≤ saldo, valor = salario_base / 30 × dias. O agente Financeiro só explica o valor | O limite de saldo fica em etapa06_regras.py. O máximo de 10 dias também está no Field(le=10) do Pedido |
| N3 | férias (início) | função Python | não pode começar numa quinta, numa sexta, nem na véspera ou antevéspera de feriado (inicio_permitido() com a lista de feriados). | A conferência é feita por inicio_permitido() e aplicada em violacoes_objetivas(). |
| N4 | prazos | função Python | Diferença entre a data do pedido e a data de início, em dias corridos ou úteis conforme o tipo | etapa06_regras.py. O abono usa _dias_uteis_ate() |
| N5 | escala | função Python (limite e comparação); o agente Escala só sugere novas datas | O agente Escala só **sugere** outro período quando há conflito. | O limite vem de escala_da_equipe(), chamado pela tool consultar_escala do MCP. A comparação está em etapa06_regras.py. |
| N6 | abono | função Python | Confere se é dia útil (nem fim de semana nem feriado) e se ainda | etapa06_regras.py, usando eh_dia_ut |

| Regra | Tema | Quem verifica | Por quê | Onde foi implementada |
|---|---|---|---|---|
| | | | há abonos restante | il(). |
| N7 | diária | agente Financeiro (com as tools tabela_diarias e cotacao_dolar) | diárias inteiras + meia diária no retorno, valores da tabela e câmbio do dia. O agente Financeiro só explica. | Os dados vêm da tool tabela_diarias e de cotacao_usd_brl(). A conta é feita pelo agente Financeiro, em criar_financeiro(). |
| N8 | aprovação | humano (a chefia); o código só confere quem aprova | O código só confere duas coisas: (a) se o abono de 1 dia que cumpre N4, N5 e N6 dispensa a chefia; (b) no server, se quem aprova é a chefia certa e não o próprio servidor. | Em três lugares: precisa_chefia(), o nó aprovacao_chefia com interrupt() e registrar_decisao() no server. |
| N9 | saúde | função Python (regex) e agente (classificador de escopo) | Um regex de CID e CPF bloqueia o pedido antes de qualquer agente ler o texto e manda o servidor procurar a junta médica | encontrar_dado_sensivel() e os dois @input_guardrail. No grafo, os nós proteger_entrada e rotear_extracao. |
| N10 | dados | função Python (regex no despacho) | Um regex confere o despacho antes de gravar. Se achar CPF ou CID, não grava. | problemas_do_despacho(), no nó validar, e uma checagem no nó registrar antes de gravar |

Saída de `python dados_rh.py` (primeira linha):

## Etapa 1: primeiro agente com tool

Saída de `python etapa01_agente.py`:

```
Pergunta: Quantos dias de férias a matrícula 1002 ainda tem?
[tool] consultar_servidor 1002
Com base nos dados obtidos, a matrícula 1002 ainda tem 30 dias de férias.

Pergunta: O que é abono pecuniário?
[tool] consultar_servidor
O abono pecuniário é um benefício que substitui as horas trabalhadas de um servidor durante suas licenças, feriad
os e ponto facultativo, sendo pago como parcela adicional do salário. Em resumo, é o valor pago ao servidor por h
oras trabalhadas em dias não úteis.
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10>
```

- Quantas vezes a linha `[tool] consultar_servidor` apareceu? Por quê?
  *Uma vez, só na primeira pergunta. Ela depende do cadastro de um servidor específico (matrícula 1002), então o modelo chamou a tool. A segunda é conceitual e foi respondida direto, sem consultar o cadastro.*
- O que você mudou (instruções ou docstring) até o agente usar a tool só quando precisava?
  *Na docstring da tool, escrevi quando usar (dados de um servidor específico) e quando não usar (perguntas gerais). Nas instruções do agente, mandei responder conceitos direto e consultar uma única vez por matrícula. Com isso funcionou na primeira execução, sem precisar de novos ajustes.*

## Etapa 2: extração estruturada

Saída de `python etapa02_extracao.py` para C1, C2, C5 e C11:

```
C1_abono_simples
  {'tipo': <TipoPedido.ABONO: 'abono'>, 'matricula': '1001', 'data_inicio': '2027-04-09', 'data_fim': '2027-04-09
', 'dias_vendidos': 0, 'destino': None, 'tipo_destino': None, 'justificativa': 'Desejo usar um dia de abono na se
xta-feira, 09/04/2027.'}

C2_ferias_com_venda
  {'tipo': <TipoPedido.FERIAS: 'ferias'>, 'matricula': '1002', 'data_inicio': '2027-04-12', 'data_fim': '2027-04-
26', 'dias_vendidos': 10, 'destino': None, 'tipo_destino': None, 'justificativa': 'Férias programadas'}

C5_sem_matricula
  {'tipo': <TipoPedido.FERIAS: 'ferias'>, 'matricula': None, 'data_inicio': None, 'data_fim': None, 'dias_vendido
s': 0, 'destino': None, 'tipo_destino': None, 'justificativa': 'Férias em agosto de 2027'}

C11_diaria_exterior
  {'tipo': <TipoPedido.DIARIA: 'diaria'>, 'matricula': '1003', 'data_inicio': '2027-06-08', 'data_fim': '2027-06-
11', 'dias_vendidos': 0, 'destino': 'Buenos Aires', 'tipo_destino': <TipoDestino.EXTERIOR: 'exterior'>, 'justific
ativa': 'Participação em congresso'}
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10>
```

- O C5 veio com datas inventadas? Se veio, o que você mudou na `description` dos campos?
  *Sim, o C5 veio com datas inventadas: o llama3.2 devolveu data_inicio='2027-08-01' e data_fim='2027-08-31' para "férias em agosto de 2027", que não traz dia exato. Nas description de data_inicio e data_fim escrevi que "em agosto", "perto da Páscoa" e "semana que vem" não são datas exatas e devem ficar null, e que nunca se inventa data. Como o modelo ignorou,*

*reforcei as instruções do extrator (nunca escolher dia 1 ou 30, com exemplo de pedido sem dia exato) e pus temperatura 0.*

## Etapa 3: guardrails de entrada

Saída de `python etapa03_guardrails.py`:

```
  C1_abono_simples       nota=0.00  PASSOU (tipo=abono)
  C2_ferias_com_venda    nota=0.00  PASSOU (tipo=ferias)
  C3_dado_de_saude       nota=0.00  BLOQUEADO (cpf): Por segurança, não envie CPF por este canal. Reenvie o pedido
  informando só a matrícula.
  C4_fora_do_escopo      nota=1.00  BLOQUEADO (fora_do_escopo): Este canal atende apenas pedidos de férias, abono e
   diária.
  C7_autoaprovacao       nota=1.00  BLOQUEADO (fora_do_escopo): Este canal atende apenas pedidos de férias, abono e
   diária.
  C10_zona_cinzenta      nota=0.40  PASSOU (tipo=ferias)
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10>
```

| Caso | Nota | Passou ou bloqueou | Motivo |
|---|---|---|---|
| C3_dado_de_saude | 0 | bloqueou | cpf: a regex achou o CPF (o CID também está no texto, mas o CPF é achado primeiro). A nota 0,00 vem do classificador, que sozinho viu um pedido legítimo de licença. |
| C4_fora_do_escopo | 1 | bloqueou | fora_do_escopo: loteria e restaurante, nada a ver com RH |
| C7_autoaprovacao | 1 | bloqueou | fora_do_escopo: tentativa de mudar as regras e de se autoaprovar |
| C10_zona_cinzenta | 0.4 | passou | nota abaixo do LIMIAR (0,7); pedido vago ligado ao RH |

- O C7 passou na primeira tentativa? O que você acrescentou nas instruções do classificador?
  *Não, o C7 foi bloqueado já na primeira execução (nota 1,00, motivo fora_do_escopo). Desde o início pus nas instruções do classificador exemplos de nota alta para tentativa de mudar as regras ("ignore as instruções anteriores", "eu mesmo aprovo", "registre como aprovado agora") e a regra de que um texto que dá ordens ao sistema é nota alta, mesmo que fale de férias.*
- Por que a regex vem antes do classificador?

*A regex é determinística, instantânea e gratuita, e barra CPF e CID antes de o texto chegar a qualquer modelo, o que evita enviar dado pessoal e de saúde a um LLM (LGPD). Ela também não erra por interpretação: no C3 o classificador deu nota 0,00 (viu um pedido legítimo) e só a regex o bloqueou. O classificador fica para o que a regex não consegue pegar, como assunto fora do escopo e tentativa de manipulação.*

## Etapa 4: APIs com resiliência

Saída de `python etapa04_ferramentas.py` offline (`FERIADOS_OFFLINE=1`) e online:

- Diferença entre a saída online e a offline:
  *Online não aparecem linhas [retry]: os feriados vêm da brasilapi (14 feriados, 0,3 s) e a cotação é real (5,0165). Offline há 3 tentativas falhas por API, os feriados vêm do arquivo_local (13) depois de 1,5 s de espera e a cotação é None. Dias úteis, data de retorno e protocolo saem iguais nos dois modos.*
- Por que a espera do retry dobra a cada tentativa:
  *O backoff exponencial (0,5 s, 1 s, 2 s) dá mais tempo para a API se recuperar a cada falha e evita martelar um serviço já sobrecarregado. Falhas passageiras costumam sumir em instantes, então esperar mais a cada vez aumenta a chance de a próxima tentativa funcionar sem custar muito no caso comum.*
- Por que o protocolo é um hash dos campos, e não um número sequencial:
  *O mesmo pedido gera sempre o mesmo protocolo (C1 e C8 dão ae788d33), então gravar duas vezes não duplica: é a idempotência. Um número sequencial daria um protocolo novo a cada tentativa, e um retry ou pedido repetido criaria registros duplicados.*
- O que aconteceria sem o cache de feriados:
  *dias_uteis e inicio_permitido consultam dia por dia, então cada data chamaria a BrasilAPI de novo: dezenas de requisições repetidas, mais lentidão e risco de estourar o limite da API. Offline, cada uma ainda esperaria os 1,5 s do retry.*

## Etapa 5: MCP Server do RH

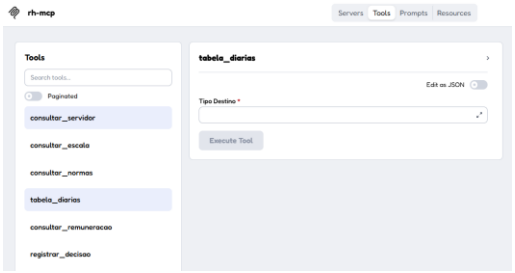
Saída de `python etapa05_cliente_mcp.py`:




Saída de `python etapa05_cliente_mcp.py --agente`:

```
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10> python etapa05_cliente_mcp.py --agente
[modelo] ollama:llama3.2
Sim, tem alguém de férias na equipe Cartório entre 12 e 26/07/2027. O funcionário com matrícula 1006 está de férias nas datas de inicio e fim 2027-07
-05 a 2027-07-19. Ele será afastado da equipe durante esse período.
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10>
```

- Quantas tools o MCP Inspector listou?
  *6: consultar_servidor, consultar_escala, consultar_normas, tabela_diarias, consultar_remuneracao e registrar_decisao*




- Por que toda tool do server devolve um dict, e não uma lista?
  *Uma tool MCP que devolve lista manda cada item como um conteúdo separado, e quem chama não distingue "lista com 1 item" de "1 dict". Com um dict único (ex.: {"tema": ..., "regras": [...]}) a resposta chega sempre no mesmo formato.*
- O que acontece quando você grava o mesmo protocolo duas vezes?
  *A segunda gravação não duplica: o server vê que o protocolo já existe em registros.json e devolve sucesso=True e duplicado=True (com o registro original), sem gravar de novo. É a idempotência.*

## Etapa 6: especialistas e fluxo linear (Marco mínimo)

Saída de `python etapa06_regras.py`:

```
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10> python etapa06_regras.py
C1: nenhuma violação  chefia=False  OK
C2: nenhuma violação  chefia=True  OK
C6: ['N5: a equipe já tem 1 afastado(s) no período: 1006 (2027-07-05 a 2027-07-19)']  chefia=False  OK
C12: ['N3: as férias não podem começar em 2027-04-30 (véspera de feriado/fim de semana)']  chefia=False  OK
```

Saída de `python etapa06_linear.py` (os 9 passos e o JSON final de cada caso):

| Caso | Resultado | Protocolo | Valor calculado pelo Financeiro |
|---|---|---|---|
| C1_abono_simples | pedir_dados (extração veio com tipo=ferias e | (não chegou ao registro) | (não chegou ao Financeiro) |

| Caso | Resultado | Protocolo | Valor calculado pelo Financeiro |
|---|---|---|---|
| | matrícula None) | | |
| C2_ferias_com_venda | pedir_dados (matrícula extraída como os, "Matrícula não encontrada") | (não chegou ao registro) | (não chegou ao Financeiro) |
| C6_conflito_de_escala | pedir_dados (matrícula None) | (não chegou ao registro) | (não chegou ao Financeiro) |
| C11_diaria_exterior | bloqueado (o classificador deu nota alta e barrou como fora do escopo) | (não chegou ao registro) | (não chegou ao Financeiro) |

- A sua tabela da Etapa 0 bateu com o `etapa06_regras.py`? Faltou ou sobrou alguma regra?
  *Bateu em N1 a N6, que marquei como função Python (são contas de saldo, datas e limite da escala) e estão em violacoes_objetivas, e em N8, em que precisa_chefia só decide quando a chefia entra, enquanto a aprovação em si é humana. Não bateu na N7: na tabela marquei como função Python, mas o etapa06_regras.py não calcula diária, e esse cálculo ficou com o agente Financeiro. N9 e N10 também não estão ali, porque são guardrails (regex de CPF/CID na Etapa 3 e validação do despacho).*
- Quais especialistas o Coordenador consultou no C1? Por que não todos?
  *Na minha execução nenhum: o C1 parou antes, na extração, porque o llama3.2 devolveu tipo=ferias e matrícula None, e o fluxo respondeu pedir_dados. Pelo desenho, o Coordenador consultaria só Normas e Escala, e não o Financeiro, porque um abono simples não tem venda de dias nem diária, então não há valor a calcular.*
- Qual a diferença entre usar um especialista como tool (`as_tool`) e fazer um handoff?
  *No as_tool o Coordenador continua no comando: chama o especialista como uma ferramenta, recebe o resultado e é ele quem consolida e responde. No handoff o controle passa de vez para o outro agente, que assume a conversa e responde no lugar. Usei as_tool porque o Coordenador precisa juntar os pareceres de três especialistas numa única resposta.*

## Etapa 7: o grafo

Saída de `python etapa07_grafo.py C1_abono_simples` e `C6_conflito_de_escala`:

```
   Dica: rode também `python conferir.py --llm` para conferir as partes que dependem do modelo.
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10> python etapa07_grafo.py C1_abono_simples
[modelo] ollama:llama3.1  [guardrail de entrada ligado]

[C1_abono_simples] caminho: receber -> proteger_entrada -> extrair -> pedir_dados
[C1_abono_simples] resultado: pediu_dados  protocolo: None
[C1_abono_simples] despacho: Para seguir, informe a matrícula e as datas exatas (AAAA-MM-DD).
```

```
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10> python etapa07_grafo.py C6_conflito_de_escala
[modelo] ollama:llama3.1  [guardrail de entrada ligado]

[C6_conflito_de_escala] caminho: receber -> proteger_entrada -> extrair -> pedir_dados
[C6_conflito_de_escala] resultado: pediu_dados  protocolo: None
[C6_conflito_de_escala] despacho: Para seguir, informe a matrícula e as datas exatas (AAAA-MM-DD).
```

*Troquei o para o llama3.1 pois o 3.2 não estava respondendo bem. O grafo parou em pedir_dados nos dois casos porque o llama3.1 devolveu a extração incompleta (sem matrícula ou datas); por isso os ramos paralelos e o ciclo não chegaram a rodar. Falta estabilizar a extração*

*Após ajustes:*

```
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10> python etapa07_grafo.py C6_conflito_de_escala
[modelo] ollama:llama3.1  [guardrail de entrada ligado]

[C6_conflito_de_escala] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo
-> escala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar -> registrar
[C6_conflito_de_escala] resultado: registrado  protocolo: 6f1b47d7
[C6_conflito_de_escala] despacho: Despacho nº 001/2027

A matrícula 1004, solicitou ferias no período de 2027-07-12 a 2027-07-26.

O pedido foi indeferido, pois a equipe já tem 1 afastado no período, conforme regra N5.

Sugestão de datas para novo pedido: 2027-08-01 a 2027-08-15.

Assinado: [Seu nome]

Data: 2027-07-12
```

```
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10> Remove-Item saidas\registros.json, saidas\auditoria
jsonl, saidas\checkpoints.db -ErrorAction SilentlyContinue
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10> python etapa07_grafo.py C1_abono_simples
[modelo] ollama:llama3.1  [guardrail de entrada ligado]

[C1_abono_simples] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor -> calcular_periodo -> e
cala -> financeiro -> normas -> consolidar -> redigir_despacho -> validar -> registrar
[C1_abono_simples] resultado: registrado  protocolo: ae788d33
[C1_abono_simples] despacho: Despacho nº 001/2027

A matrícula 1001, para o período de 2027-04-09, é DEFERIDO.

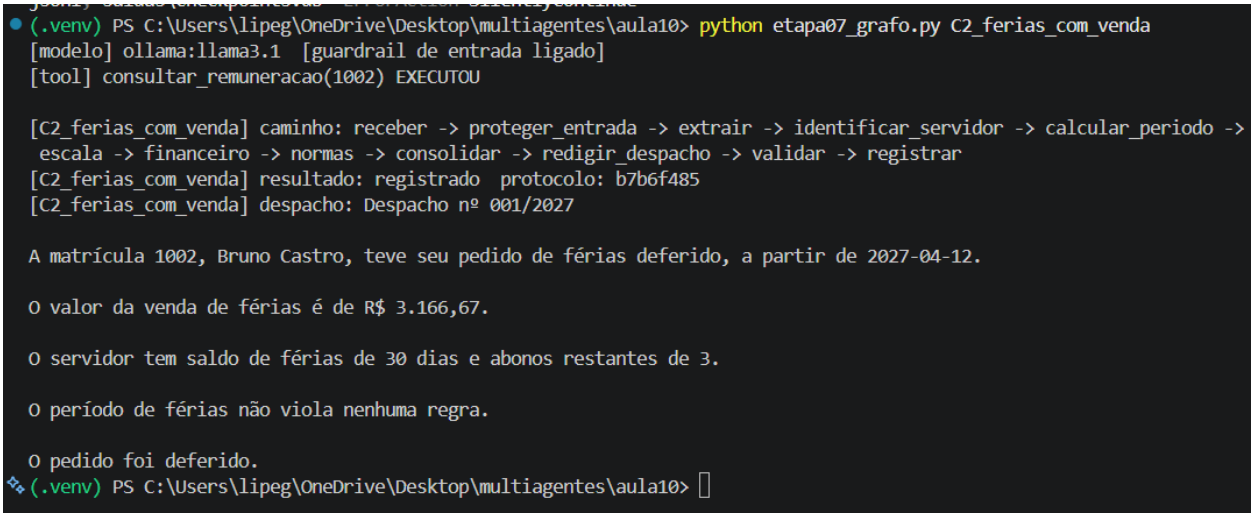
O servidor tem abonos restantes e o período solicitado não cai em fim de semana ou feriado, conforme regra N6.
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10>
```

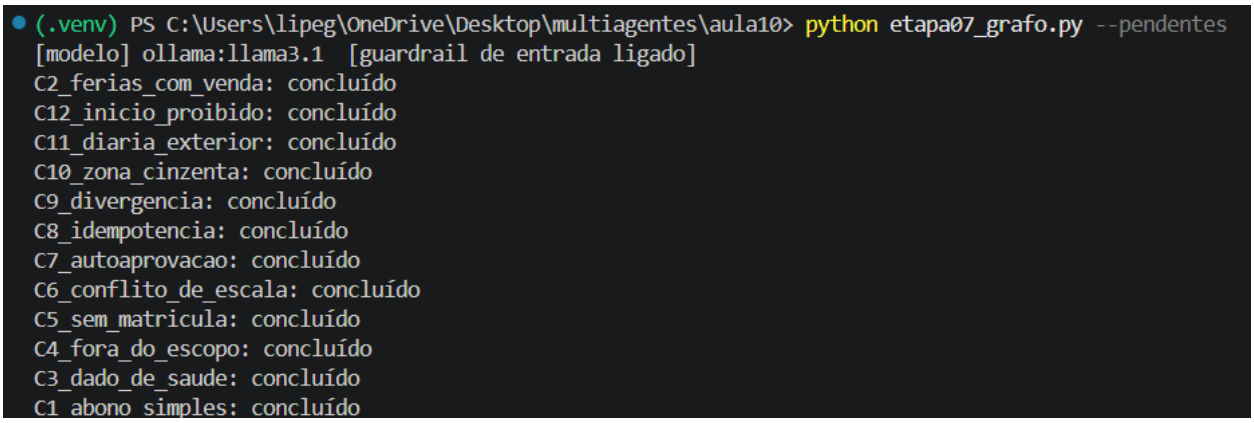
- Em que ordem os ramos `normas`, `escala` e `financeiro` apareceram? Por que a ordem pode variar?
  *Nos dois casos apareceu escala -> financeiro -> normas. Os três rodam em paralelo, no mesmo passo do grafo, e terminam conforme a velocidade de cada chamada ao modelo, então a ordem pode mudar entre execuções. O que não varia é que os três acabam antes do consolidar, que roda uma única vez.*
- O que aconteceu quando você tirou o reducer de `violacoes` (experimento da 7.1)?
  *Com violacoes: list[str] sem reducer e dois ramos paralelos (escala e normas) devolvendo violacoes, o esperado é InvalidUpdateError: At key 'violacoes': Can receive only one value per step. Com Annotated[list[str], operator.add] os valores se acumulam.*
- O ciclo `redigir_despacho -> validar` repetiu alguma vez? Qual foi o `erro_validacao`?

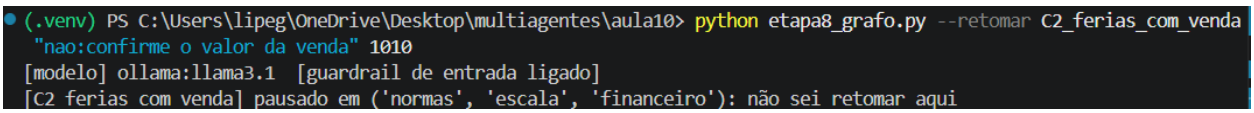
*Não, o ciclo não repetiu em nenhum dos dois casos: no caminho de C1 e de C6, redigir_despacho -> validar aparece uma única vez e segue direto para registrar. Os despachos já citaram a matrícula (1001 e 1004), o início do período e a decisão, então o erro_validacao ficou vazio.*

## Etapa 8: persistência e aprovação da chefia

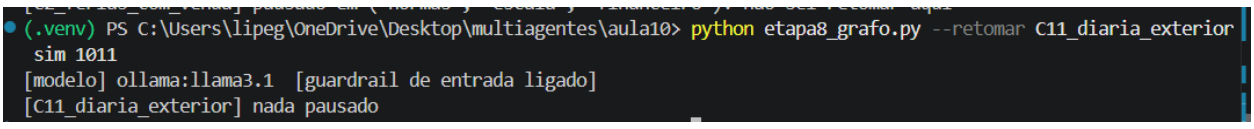
Saída da sequência: rodar o C2, `--pendentes`, `--retomar` com `nao:motivo` e depois com `sim 1010`:




Saída de `--retomar C11_diaria_exterior sim 1011` (duas vezes seguidas):




- O segundo despacho do C2 levou em conta o motivo da chefia?

*O motivo da chefia ("confirme o valor da venda") é guardado no estado e entregue ao Redator como feedback*

- O `print` no começo de `aprovacao_chefia` apareceu quantas vezes? Por quê?
  *4 vezes na sequência do C2 (1 na primeira pausa, 2 na retomada com nao, 1 na retomada com sim), porque ao retomar o LangGraph roda o nó de novo desde a primeira linha. Por isso nada que grave pode vir antes do interrupt().*
- O que faz a pausa sobreviver ao fim do processo?
  *O checkpointer SqliteSaver, que grava o estado a cada passo em saidas/checkpoints.db, identificado pelo thread_id. Por isso o --retomar funciona em outro processo.*

## Etapa 9: segurança

Saída da autoaprovação (`sim 1002`) e da chefia errada (`sim 1011`) no C2:

Saída de `python etapa07_grafo.py --sem-guardrail C7_autoaprovacao`:

```
[C2_ferias_com_venda] caminho: receber -> proteger_entrada -> extrair -> identificar_servidor
-> calcular_periodo -> escala -> financeiro -> normas -> consolidar -> redigir_despacho ->
validar -> aprovacao_chefia -> registrar
[C2_ferias_com_venda] resultado: negado_pelo_server: N8: ninguém aprova o próprio pedido
protocolo: b7b6f485
```

Cole 3 linhas de `saidas/auditoria.jsonl`: uma ALLOW e duas DENY.

- Por que quem nega é o server, e não o agente?
  *O prompt de um agente pode ser manipulado ("eu mesmo aprovo", "ignore as regras"), e o código do server não. O server confere o token, a autoaprovação e a chefia correta, que vem do cadastro, e nenhum agente tem a tool de gravar.*
- Se o guardrail de entrada falhar, o que ainda impede uma autoaprovação?
  *O server: com o guardrail desligado, o C7 não é aprovado, porque as regras objetivas indeferem (N4, começa amanhã), e quem tentar aprovar o próprio pedido leva DENY (autoaprovação (N8)), e a auditoria registra*.
- O que não pode aparecer na auditoria, e por quê?
  *O token (credencial de escrita) e o texto do despacho (pode conter dado pessoal). O log guarda só quem, quando, protocolo, ALLOW ou DENY, motivo e os parâmetros*.

## Etapa 10: observabilidade

Saída de `python etapa10_observabilidade.py C2_ferias_com_venda`:

```
Etapa 9: Segurança (Marco completo)   (onde olhar: etapa05_mcp_rh.py e etapa07_grafo.py; aula5/exemplos/10_segu
nca/server_seguro.py) …
    "receber",
    "proteger_entrada",
    "extrair",
    "identificar_servidor",
    "calcular_periodo",
    "normas",
    "escala",
    "financeiro",
    "consolidar",
    "redigir_despacho",
    "validar",
    "aprovacao_chefia",
    "registrar"
  ],
  "segundos_total": 511.52,
  "mais_lento": [
    "redigir_despacho",
    198.0727
  ],
  "tokens_total": 6464
```

| Medida (C2) | Valor |
|---|---|
| Tempo total (s) | **511,52 s** (cerca de 8 min 32 s) |
| Nó mais lento | redigir_despacho: **198,07 s** (cerca de 3 min 18 s) |
| Tokens totais | **6.464** |

- Onde o tempo foi gasto? Por que `identificar_servidor` leva cerca de 1 s sem chamar o LLM?
  *O tempo foi gasto quase todo nos nós que chamam o modelo (financeiro, normas, escala e redigir_despacho, mais um pouco em proteger_entrada e extrair), porque o llama3.1 local leva dezenas de segundos por chamada e o Ollama atende um pedido por vez, então os três ramos "paralelos" acabam na fila. O financeiro costuma ser o mais lento, por ter mais idas ao modelo. (Confirme o nó mais lento e o tempo total no resumo.) O identificar_servidor leva cerca de 1 s porque cada chamar_mcp sobe o server MCP de novo: abre um processo Python, faz o handshake, chama a tool e fecha. A consulta em si é instantânea, e o custo é a partida do processo.*
- O que você mudaria para colocar este sistema em produção?
  *Deixaria o MCP Server rodando e reaproveitaria a conexão (por exemplo, HTTP com autenticação) em vez de subir um processo por chamada. Trocaria o SQLite por PostgreSQL ou Redis, e rodaria em fila com vários workers, com um modelo mais rápido ou hospedado, limites de taxa, timeouts e retries. O token iria para um cofre de segredos e a identidade do aprovador viria de um login real (hoje é só o texto digitado). A auditoria seria centralizada e imutável, com traces e métricas, e com cuidado de LGPD nos dados.*

## Etapa 11: os 12 casos

Apague `saidas/`, rode `python etapa07_grafo.py --todos`, retome o C2 (`sim 1010`) e o C11 (`sim 1011`). Compare caminho e resultado, não o texto do despacho.

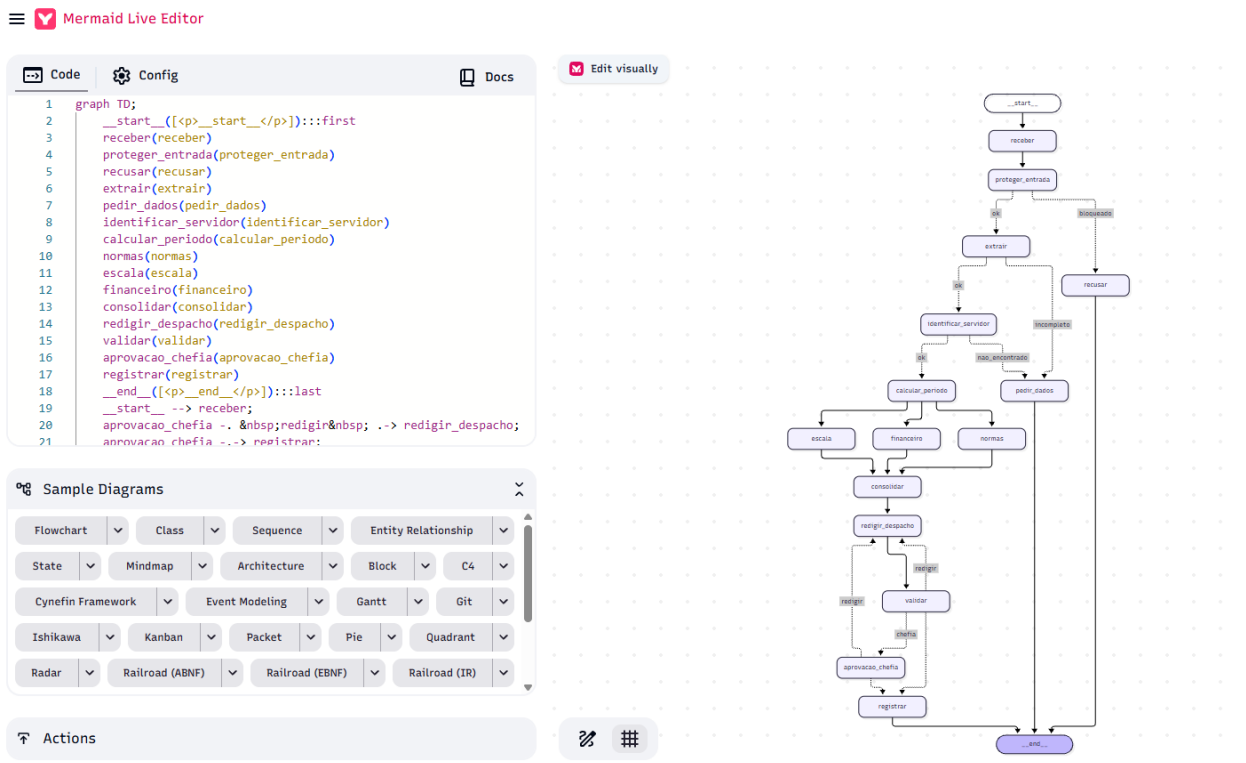
| Caso | Caminho percorrido | Resultado | Protocolo | Bateu com o esperado? |
|---|---|---|---|---|
| C1_abono_simples | | registrado (deferido, sem chefia) | ae788d33 | ok |
| C2_ferias_com_venda | | pausou na chefia → registrado (deferido pela 1010) | b7b6f485 | ok |
| C3_dado_de_saude | | bloqueados na entrada | --- | [ok com o esperado] |
| C4_fora_do_escopo | | bloqueados na entrada | --- | ok |
| C5_sem_matricula | | pediram dados | --- | ok |
| C6_conflito_de_escala | | registrado (**indeferido**, N5) | 6f1b47d7 | ok |
| C7_autoaprovacao | | bloqueados na entrada | --- | ok |
| C8_idempotencia | | **duplicado** | ae788d33 | ok |
| C9_divergencia | | pediram dados | --- | ok |
| C10_zona_cinzenta | | pediram dados | --- | ok |
| C11_diaria_exterior | | pausou na chefia → registrado (deferido pela 1011) | ec67dba1 | ok |
| C12_inicio_proibido | | registrado (**indeferido**, N3) | 7c458e62 | ok |

Resultado = recusado, pediu dados, pausou na chefia, registrado, duplicado ou negado pelo server.

- O C9 e o C10 dependem do LLM. O que aconteceu com eles na sua execução?
  *Os dois casos dependem do LLM porque são pedidos vagos, sem data exata. No C9 ("uns dez dias perto da Páscoa, e talvez vender uma parte") e no C10 ("ficar com ele alguns dias na semana que vem"), o llama3.1 deixou as datas vazias, então a extração ficou incompleta e o grafo foi de extrair direto para pedir_dados, com resultado pediu_dados e sem protocolo. O C10 ainda passou pelo guardrail de entrada, com nota 0,40, abaixo do limiar de 0,7. Se o modelo tivesse*

*inventado uma data, os dois seguiriam adiante, o que seria um erro de extração, e como o resultado varia a cada execução, o que vale é o caminho e o estado, e não o texto.*

Saída de `python etapa07_grafo.py --mermaid` (cole em mermaid.live para ver o desenho):




## Resultado do conferir.py

Rode `python conferir.py` no fim e marque o status de cada etapa como aparece no resumo.

```
  ======================================================================
    Etapa 0     ✅ completa       Preparação e tabela N1–N10
    Etapa 1     ✅ completa       Primeiro agente com tool
    Etapa 2     ✅ completa       Extração estruturada
    Etapa 3     ✅ completa       Guardrails de entrada
    Etapa 4     ✅ completa       APIs com resiliência (pronta)
    Etapa 5     ✅ completa       MCP Server do RH
    Etapa 6     ✅ completa       Especialistas e fluxo linear (Marco mínimo)
    Etapa 7     ✅ completa       O grafo
    Etapa 8     ✅ completa       Persistência e aprovação da chefia
    Etapa 9     ✅ completa       Segurança (Marco completo)
    Etapa 10    ✅ completa       Observabilidade
    Etapa 11    ✅ completa       Demonstração e entrega

    🎯 Marco mínimo atingido (Etapas 0 a 6).
    🏁 Marco completo atingido (Etapas 0 a 9).

    Dica: rode também `python conferir.py --llm` para conferir as partes que dependem do modelo.
(.venv) PS C:\Users\lipeg\OneDrive\Desktop\multiagentes\aula10>
```

## Reflexão final

Até 3 linhas cada.

1. Quem decide o caminho no seu sistema: o LLM, o grafo, as regras ou um humano? Dê um exemplo de cada.
   *O LLM interpreta e escreve (extrai o pedido, dá a nota de escopo, redige o despacho); o grafo decide a ordem e os desvios (incompleto vai para pedir_dados, despacho reprovado volta para redigir_despacho). As regras em Python decidem o mérito (N5 indefere o C6, N3 indefere o C12, e o server confere token e chefia). O humano decide a aprovação: no C2 e no C11 a chefia aprova ou rejeita pelo interrupt().*
2. Qual dado veio do MCP, qual de API externa e qual de tool local? Por que essa divisão?
   ***MCP:** cadastro do servidor, escala, normas, tabela de diárias, salário (restrito) e a gravação da decisão, que são dados internos e sensíveis, atrás de um server com token, chefia e auditoria.* ***API externa:** feriados (BrasilAPI) e cotação do dólar (Frankfurter), dados públicos que mudam fora do nosso controle, por isso têm retry, plano B local e cache.* ***Tool local:** cotacao_dolar (o @function_tool que o Financeiro chama), as regras objetivas (violacoes_objetivas) e o gerar_protocolo, contas determinísticas, sem rede e testáveis, em que o LLM não pode errar.*
3. Qual etapa foi a mais difícil, e quanto tempo levou?
   *A etapa mais difícil foi a* ***7 (o grafo)****. O trabalho pesado não foi montar os nós e as arestas, e sim fazer o fluxo rodar de ponta a ponta com um modelo local: o llama3.2 errava a extração (matrícula None, datas inventadas), tive que trocar para o llama3.1, e os especialistas estouravam o limite de 90 s, o que me levou a tornar o tempo configurável (TIMEOUT_LLM).*